Universidade do Porto

Faculdade de Engenharia

FEUP

Projeto Final - Manipulador

João Nuno Borges Ruivo
Pedro Miguel Oliveira Rodrigues

Relatório do Projeto Final realizado no âmbito da Unidade Curricular de Arquiteturas de Computação Embarcada, do Mestrado em Engenharia Eletrotécnica e de Computadores

Data de entrega: 04-02-2024

Declaramos que o presente trabalho/relatório é da nossa autoria e não foi utilizado previamente noutro curso ou unidade curricular, desta ou de outra instituição. As referências a outros autores (afirmações, ideias, pensamentos) respeitam escrupulosamente as regras da atribuição, e encontram-se devidamente indicadas no texto e nas referências bibliográficas, de acordo com as normas de referenciação. Temos consciência de que a prática de plágio e autoplágio constitui um ilícito académico.

In "Código Ético de Conduta Académica", art.14, Universidade do Porto.

1 Resumo

Este relatório consiste na exposição das estratégias adotadas, com recurso a um manipulador impresso em 3D, um sensor TOF (Vl53l0x), um sensor RGB (TCS34725), 4 servo motores e um Raspberry Pi Pico, de forma a conseguir que o manipulador, sob 3 cenários distintos, consiga analisar objetos de cores diferentes e organizá-los de acordo com a sua cor.

Índice

1 Resumo **1**

Índice **2**

2 Introdução **3**

3 Cinemática Inversa **5**
3.1 Equações da Cinemática Inversa 5
3.2 Calibração dos servomotores 6

4 Sensorização **6**
4.1 Sensor RGB 6
4.2 Sensor de tempo de voo (*Time of Flight*) 7

5 Fluxo de Controlo **7**
5.1 *Input* do Utilizador e Emissão de Eventos 7
5.2 Conceito de Tarefa 9

6 Máquina de Controlo Principal **9**

7 Máquina de Movimento Suave **10**

8 Máquina de *Pick and Drop* **12**

9 Máquina de recolha de uma matriz 3x3 . . . . . **13**

10 Máquina de varrimento com auxílio de sensor de tempo de voo **14**

11 Estruturação Código e Implementação **15**

12 Conclusão **15**

2 Introdução

Este trabalho consistiu na criação de um manipulador funcional com recurso ao microcontrolador RP2040 presente na placa Raspberry Pi Pico com o auxílio de 2 sensores e 4 servo motores com funcionamento coordenado para se obter o movimento desejado. O esquemático do circuito utilizado encontra-se representada na figura 1 e a montagem do manipulador encontra-se representada na figura 2.

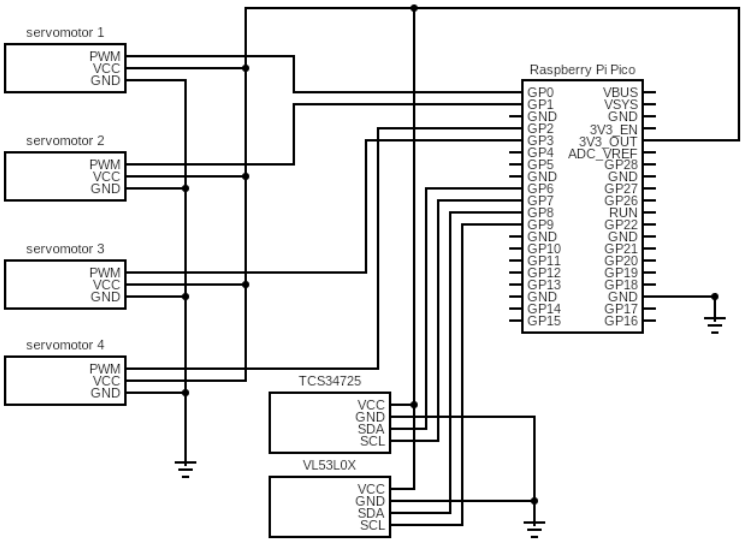

Figura 1: Esquemático do circuito eléctrico

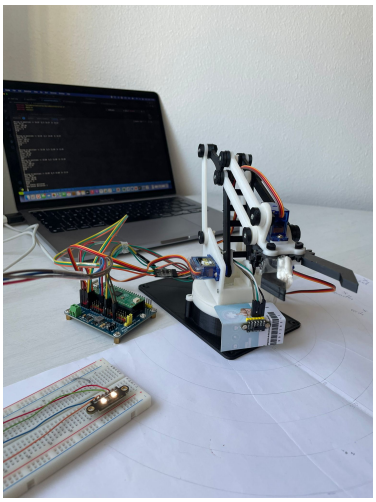

Figura 2: Imagem da montagem do manipulador

Inicialmente foram adquiridos todos os componentes necessários, tais como os servomotores, os sensores TOF e RGB e o shield para o Raspberry Pi Pico.

Após isso, procedeu-se à montagem do robô com as peças fornecidas.

Identificada a configuração e tipo de juntas utilizadas, calculou-se a cinemática inversa para o manipulador, que será posteriormente explorada em mais detalhe.

Implementada a cinemática inversa e os comandos que permitem ao utilizador interagir com o manipulador através do terminal, alcançou-se a capacidade de movimento pretendida e procedeu-se à integração dos 2 sensores externos (VL53l0x e TCS34725).

Numa primeira fase, deliberou-se que seria importante colocar o sensor TOF numa posição central e baixa, de forma a que pudesse eficazmente detetar as peças usadas neste contexto. Para tal, com recurso a um cartão bancário velho, colou-se o sensor numa posição mais eficiente - junto ao próprio manipulador, como pode ser observado na figura 2.

Numa segunda fase, integrou-se o sensor RGB. Para tal, e sabendo que este teria de estar numa posição fixa (armazenada no manipulador), montou-se o mesmo numa *breadboard*, colocada lateralmente ao manipulador.

Assim, com tudo funcional, o manipulador deveria ser capaz de executar as seguintes tarefas:

- Receber a informação da posição de uma peça, pegar nela e identificar a sua cor e por fim largá-la no respetivo contentor, cuja posição também deve ser capaz de receber e guardar. Idealmente, o manipulador deve ser capaz de executar esta tarefa continuamente, sendo possível alterar os parâmetros (posição da peça, posição do contentor, sensor de cor) continuamente, sem recompilar o programa ou até mesmo guardando em memória;

- Recolher, analisar e depositar corretamente todas as peças de uma matriz 3x3 (num total de 9), cujas posições podem ser configuradas pelo utilizador;

- Recolher, analisar e depositar corretamente uma peça cuja posição é previamente desconhecida pelo manipulador.

O funcionamento do sistema como um todo ficou organizado em 5 máquinas de estado, responsáveis por partes diferentes, tendo sido usado um sistema de *flags* para gerir as comunicações entre todas as máquinas.

3 Cinemática Inversa

Um dos desafios deste trabalho é descobrir as posições angulares dos servomotores de modo que a garra do manipulador atinja a posição desejada.

As posições angulares dos servomotores foram determinadas a partir da cinemática inversa do manipulador.

A cinemática inversa envolve descobrir os ângulos das juntas do manipulador com base nas coordenadas da posição desejada da garra. Este manipulador possui três juntas rotacionais, uma na base e as outras duas no braço. O esquema simplificado encontra-se apresentado na figura 3.

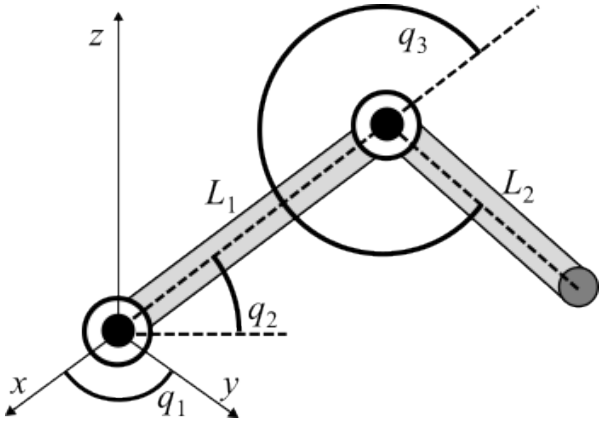

Figura 3: Esquema do manipulador 3R com junta cilíndrica utilizado.

Visto que a primeira junta do manipulador é cilíndrica, as coordenadas mais apropriadas para representarem o seu movimento são as coordenadas cilíndricas (r, ϕ, z). A primeira junta (q_1) controla a posição ângular do manipulador (ϕ) enquanto que a segunda e terceira juntas (q_2 e q_3) controlam a distância radial (r) e a altura da garra (z). Os elos do manipulador L_1 e L_2 têm ambos o mesmo comprimento cujo valor é 8 cm.

3.1 Equações da Cinemática Inversa

Com base na posição desejada da garra (r, ϕ, z) determina-se os ângulos das juntas (q_1, q_2 e q_3) a partir das equações enunciadas abaixo (derivadas com base na referência [1]).

O processo inicia-se ao configurar a primeira junta (q_1) de modo a alinhar os elos do manipulador com a posição final desejada para a garra, obtendo-se assim a equação 1.

De seguida, determina-se a posição da terceira junta (q_3), através da equação 2, e depois a da segunda junta (q_2), uma vez que está depende da posição da terceira, como apresentado na equação 3.

$$q_1(\phi) = \phi \tag{1}$$

$$q_3(r, z) = \pm 2 \arctan\left(\sqrt{\frac{(2L)^2}{r^2 + z^2} - 1}\right) \tag{2}$$

$$q_2(r, z) = \operatorname{atan2}(z, r) - \operatorname{atan2}\left(L \sin\left(q_3\right), L\left(1 + \cos\left(q_3\right)\right)\right) - \frac{\pi}{2} \tag{3}$$

Tendo em conta os aspetos mecânicos do manipulador, o ângulo da terceira junta (q_3) tem valor negativo e por este motivo na equação 2 considerou-se sempre a solução negativa. O valor do ângulo da segunda junta (q_2) já pode tomar um valor tanto positivo como negativo.

Através da movimentação dos elos do manipulador, foi possível compreender quais eram os limites do movimento e com base nessas posições limite definiu-se intervalos para as coordenadas cílindricas válidas para o posicionamento da garra do manipulador. Desta forma, tentou-se não forçar a estrutura frágil do manipulador.

3.2 Calibração dos servomotores

De modo a controlar a posição das juntas utilizou-se três servomotores. Além destes três servomotores existe outro servomotor responsável por controlar a abertura e fecho da garra do manipulador. Cada servomotor foi identificado por um número de 1 a 4 e a posição ângular de cada um é identificada pela variável θ_i.

A partir da análise das características mecânicas do manipulador determinou-se a relação entre a posição angular dos servomotores 1, 2 e 4 e a posição angular das juntas do manipulador que se encontram nas equações abaixo.

$$\theta_4 = q_1; \qquad \theta_2 = -q_2; \qquad \theta_1 = 180^\circ - \theta_2 - q_3. \tag{4}$$

Os servomotores utilizados têm uma amplitude de movimento de 180°, ou seja, $\theta_i \in [-90^\circ, +90^\circ]$, e são controlados com o auxílio de um sinal períodico PWM (*Pulse Width Modulation*), sendo a sua posição angular controlada pelo *duty cycle* do sinal.

Cada servomotor tem um valor mínimo e máximo de *duty cycle* que vão corresponder às posições angulares limite do seu movimento. Esses valores limite foram caracterizados para cada servomotor e com base neles efetuou-se uma regressão linear, obtendo assim uma relação entre o *duty cycle* e a posição angular dos servomotores. A importância desta calibração baseia-se no facto de que se os valores de *duty cycle* excederem os valores limites do servomotor, a probabilidade deste se danificar aumenta consideravelmente.

4 Sensorização

4.1 Sensor RGB

Utilizou-se o sensor TCS34725 para se aferir a cor das peças que o manipulador agarrava, através dos valores medidos para cada componente RGB. Com base nestes, podemos separar as peças de acordo com a sua cor. Era sabido de antemão que as peças a utilizar tinham 4 cores: vermelho, verde, azul e amarelo.

Primeiramente, é importante referir as dificuldades desta medição e que pequenas variações de distância da peça ao sensor alteram significativamente os valores da medição mesmo esta sendo normalizada ao ser dividida pelos counts do canal *clear*.

Também se testou os valores HSV mas acabou por se considerar apenas as razões entre os canais *Red*, *Green* e *Blue* já que a utilização do sistema HSV não demonstrou francas melhorias relativamente às medições obtidas anteriormente e consequente distinção de cores.

Quanto à implementação deste sensor, analisou-se inicialmente o código exemplo fornecido no Moodle. Nesse projeto era criada a função *getRawData_noDelay* que, contrariamente à função análoga da biblioteca da Adafruit, não faz um *delay()* após cada medição do sensor, já que aqui não seria necessário. Esta função é então responsável pela leitura dos valores para cada canal, incluindo o *clear*.

Para normalizar os valores medidos e colocá-los na escala de 0-255, criou-se a função *get_ color_ data*, que dividia cada valor R, G e B pelos counts do *clear* e multiplicava por 255.

De seguida, e dada as incongruências constantes nas medições (principalmente para a peça azul) e sendo difícil garantir que as medições feitas já em conjunto com o manipulador aconteciam nas condições "ideais", optou-se por analisar as relações entre componentes R, G e B para diferenciar as peças. Uma predonimância do valor de *RED* face a *GREEN*, numa razão superior a 1,25 indicaria com segurança uma peça vermelha. Uma razão inferior a esse valor representaria uma peça amarela. Uma predominância do valor de *GREEN* relativamente a *RED* e *BLUE* seria uma peça verde. Um valor *BLUE* superior a *GREEN* ou um cenário diferente dos já mencionados seria interpretado como peça azul.

Finalmente, quando incorporado no manipulador, o sensor seria posicionado na lateral do manipulador, com recurso a uma *breadboard* externa e a sua posição seria dada ao manipulador, sendo que este realizaria uma medição sempre que agarrasse uma peça e despejá-la-ia no contentor correspondente, cuja posição também lhe era fornecida e guardada.

4.2 Sensor de tempo de voo (*Time of Flight*)

Utilizou-se o sensor VL53l0x para se medir a distância desde a base do manipulador até à peça que se pretende que o manipulador alcance.

Em cada movimento do manipulador são feitas 3 medições, calculando o valor médio correspondente de forma a diminuir o erro da medição de distância.

É importante notar que peças com uma pequena área de superfície, como é o caso das peças quando colocadas lateralmente, tendem a dar leituras mais incertas, com maior erro de medição e muitas vezes com falha de deteção.

Além disso, é relevante mencionar também que quando se tem múltiplas peças juntas torna-se difícil distinguir as peças.

Para se encontrar a posição angular das peças temos um vetor/*array* com os valores de distância por cada posição angular. Depois realiza-se um filtro de médias móveis com uma janela de três pontos para se tentar suavizar os dados, de seguida usa-se um algoritmo simples para se encontrar os mínimos locais dos dados. Os mínimos locais vão corresponder às peças, portanto, a partir do ângulo do mínimo local e da distância desse mínimo local sabemos nos alinhar com a peça e a que distância radial temos que colocar a garra para apanhar a peça.

5 Fluxo de Controlo

Com o objetivo de desenvolver um manipulador altamente configurável, pronto a responder aos comandos do utilizador e capaz de executar diversas tarefas, foi crucial a implementação de uma biblioteca responsável pelo *input* do utilizador através do terminal, onde se incorporou o conceito de eventos. Para se obter uma boa gestão da execução de múltiplas tarefas, introduziu-se o conceito de tarefa, o qual será detalhadamente abordado na subseção seguinte.

5.1 *Input* do Utilizador e Emissão de Eventos

A biblioteca utilizada para a gestão dos comandos enviados pelo utilizador foi a biblioteca *commands.h* disponibilizada na página do Moodle da unidade curricular.

Esta biblioteca processa os comandos (letra) e valores associados (números). Após o processamento do comando e do valor associado, uma função externa à biblioteca (*process_command*) é chamada e ,consoante o comando, uma ação é realizada dentro dessa função.

Os comandos disponíveis (apresentados na tabela 1) estão divididos em comandos responsáveis pelo movimento do manipulador a pedido do utilizador, comandos que alteram o estado de funcionamento do manipulador (comandos de controlo), comandos de medição e comandos de configuração.

Comandos	Tecla	Funcionalidade
movimento	q*	altera o valor de r
	w*	altera o valor de ϕ
	e*	altera o valor de z
	r	emite evento *flag_smooth_move*
	t	decrementa r
	y	incrementa r
	u	decrementa ϕ
	i	incrementa ϕ
	o	decrementa z
	p	incrementa z
controlo	z	emite evento *flag_pick_drop_start*
	x	emite evento *flag_tof_scan_start*
	l	emite evento *flag_matrix_start*
	b	emite evento *flag_stop_tof* ou *flag_matrix_stop*
	c	entrar/sair modo configuração
medição	h	imprime a distância atual do sensor TOF
	j	imprime a cor atual do sensor RGB
	v	imprime a posição atual
configuração	a	guarda a posição do depósito verde
	s	guarda a posição do depósito vermelho
	d	guarda a posição do depósito azul
	f	guarda a posição do depósito amarelo
	g	guarda a posição do sensor RGB
	z*	altera o valor mínimo do ângulo do varrimento com o sensor TOF
	x*	altera o valor máximo do ângulo do varrimento com o sensor TOF
	l*	guarda na n-éssima entrada da matriz a posição atual, onde n = {1,...,9}

* estes comandos têm que ser precedidos por um valor númerico

Tabela 1: Comandos que o utilizador pode fornecer ao manipulador através do terminal e respetivas funcionalidades.

Os comandos de controlo emitem eventos. Um evento é caracterizado por uma variável do tipo *int* com prefixo *flag* responsável por sinalizá-lo. Quando o evento é produzido, esta variável passa a valer 1 e quando esta é observada/consumida o seu valor é alterado para 0. A responsabilidade de consumir o evento (trocar o valor da variável para zero) é da porção de código que o observa. Estes eventos são responsáveis por ativar as diferentes tarefas.

5.2 Conceito de Tarefa

Uma tarefa define-se pela execução de uma máquina de estados, iniciando-se no primeiro estado (estado 0 ou *Maq Off*) e progredindo através de vários estados até ser completada, isto é, a tarefa é concluída quando se retorna ao estado inicial. Para se iniciar a tarefa, é necessário ser produzido um evento que será consumido pela máquina de estados.

Cada estado representa uma fase específica no ciclo de execução da tarefa.

Ao progredir através dos estados, a tarefa realiza diversas operações tal como iniciar outra tarefa através da produção de um evento.

A conclusão da tarefa é sinalizada ao produtor do evento que a iniciou através da verificação de duas condições: o estado da máquina de estados responsável pela execução da tarefa voltou a ser o estado 0; e o tempo decorrido desde a emissão do evento é superior a três ciclos de execução do programa ($3 \times 40\,\text{ms}$). Esta última condição é necessária para garantir que a tarefa não é dada como terminada pelo produtor do evento se após um ciclo de 40 ms (valor da variável *interval*) a máquina de estados da tarefa não teve oportunidade de sair do estado inicial.

6 Máquina de Controlo Principal

De modo a controlar o modo de funcionamento do manipulador desenvolveu-se uma máquina de estados responsável pelo controlo de alto nível do comportamento do manipulador. Essa máquina encontra-se representada na figura 4:

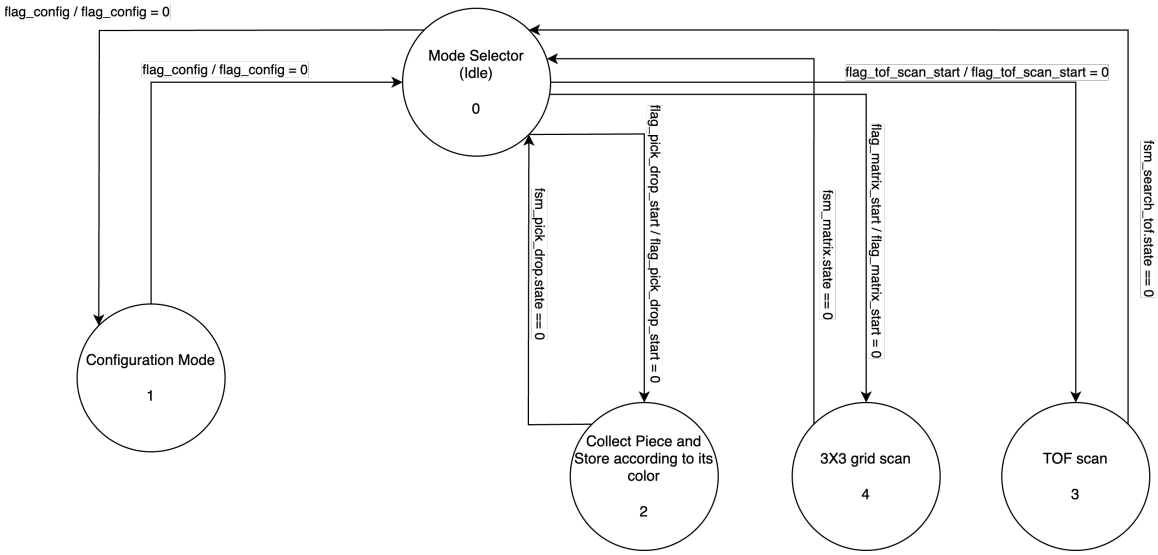

Figura 4: Máquina de Controlo Principal.

Esta máquina de estados serve como intermediário entre o utilizador e o manipulador uma vez que recebe os inputs do utilizador através dos eventos que estes produziram na função *process_ command* e cria eventos que darão início às tarefas de cada estado principal de funcionamento. Além disso, esta máquina indica e controla o estado principal de funcionamento em que o manipulador se encontra.

Através dos estados da máquina de controlo principal (figura 4) percebe-se que os estados principais de funcionamento do manipulador são o modo de configuração, o modo em que

o manipulador realiza a tarefa de *pick and drop*, o modo em que o manipulador apanha as peças de uma grelha pré configurada 3 por 3 e o modo em que o manipulador realiza um varrimento com um sensor de tempo de voo para detetar as peças e posteriormente apanhar as mesmas.

Os estados 2, 3 e 4 são estados que representam a ativação e execução das três tarefas principais do manipulador enquanto o estado 1 representa o modo de configuração que consiste simplesmente na mudança das funcionalidades dos comandos que o utilizador pode inserir apresentados na secção da configuração da tabela 1.

7 Máquina de Movimento Suave

O movimento de uma posição $\vec{r}_i$ para outra posição $\vec{r}_f$ é executado com uma velocidade linear constante e uma velocidade angular constante. Quando se quer fazer um movimento este é considerado como a execução de uma tarefa, tarefa esta que é executada pela máquina de estados representada na figura 5.

Esta tarefa é ativada através do evento *flag_smooth_move* e utiliza a informação contida na estrutura de dados *target_pos* (estrutura do tipo *Coordinates* que contêm três floats com as três componentes de uma coordenada) para saber qual é a posição final onde o manipulador se deve encontrar. Ao longo dos movimentos, a estrutura de dados *curr_pos* (estrutura do tipo *Coordinates*) vai sendo sempre atualizada com a posição atual do manipulador.

Durante a execução desta tarefa, o movimento pode ser composto, no máximo, por três etapas: retração, alinhamento e extensão.

A etapa de retração é importante para se garantir que quando se realiza o alinhamento os elos do manipulador não colidem com as peças que possam estar no seu caminho. Portanto, quando esta etapa é executada, o manipulador retrai, colocando a garra numa posição recolhida e elevada de modo a que ao rodar esta não colida com as peças.

Caso o manipulador não esteje alinhado com a posição final nem esteja numa posição segura, vai primeiramente retrair para uma posição segura (estado 1), de seguida alinhar-se com a posição final (estado 2) e no fim extender-se até essa posição (estado 3). Quando esta etapa termina a máquina regressa ao estado inicial.

Caso o manipulador já esteja numa posição segura apenas necessita de executar o alinhamento (transita do estado 0 para o estado 2 diretamente) e depois executa a extensão.

E, por fim, caso o manipulador já esteja alinhado com a posição final, não necessita de alinhamento nem de regressar à posição segura pelo que executa apenas um movimento linear até à posição final.

Os movimentos de retração e extensão foram definidos como movimentos lineares com uma velocidade linear constante. De forma a estes serem realizados de forma suave foram criadas duas estruturas de dados auxiliares do tipo *Coordinates*: *initial_pos* onde é guardada a posição inicial de uma etapa do movimento e *intermediate_pos* onde é armazenada as posições intermédias da etapa do movimento.

Para a etapa de retração, a posição final será uma posição com as coordenadas (r, z) da posição segura mas mantendo a mesma posição angular ϕ. No caso da etapa de alinhamento a posição objetivo dessa etapa do movimento terá os mesmos valores de (r, z) mas o valor da coordenada ϕ será igual ao ângulo de alinhamento final. Já na etapa de extensão as coordenadas objetivo serão as armazenadas na estrutura de dados *target_pos*.

Para tal, vamos definir algumas quantidades úteis às etapas de retração e extensão: a velocidade linear do movimento v_{linear}, a distância que tem que ser percorrida segundo a

componente r numa etapa do movimento, $\Delta r = r_f - r_i$ e a distância que tem que ser percorrida segunda a componente z, $\Delta z = z_f - z_i$. Deste modo, a distância linear do movimento de uma etapa é dada por $\Delta\vec{r} = \sqrt{\Delta r^2 + \Delta z^2}$ e o tempo necessário para a execução do movimento linear é dado por $\Delta t = \frac{\Delta\vec{r}}{v_{linear}}$. Através do tempo em que estamos no estado, tis, controla-se a posição atual do movimento através das equações 5 e 6.

$$r(tis) = \begin{cases} r_i + \frac{\Delta r}{\sqrt{\Delta r^2 + \Delta z^2}} v_{linear} tis & \text{, se } tis \leq \Delta t \\ r_f & \text{, se } tis > \Delta t \end{cases} \tag{5}$$

$$z(tis) = \begin{cases} z_i + \frac{\Delta z}{\sqrt{\Delta r^2 + \Delta z^2}} v_{linear} tis & \text{, se } tis \leq \Delta t \\ z_f & \text{, se } tis > \Delta t \end{cases} \tag{6}$$

Os movimentos de alinhamento são simplesmente movimentos circulares com uma velocidade angular constante $\omega_{alinhamento}$. De forma análoga, definiu-se a distância ângular do movimento como $\Delta\phi = \phi_f - \phi_i$ pelo que o tempo necessário para a execução deste movimento é dado por $\Delta t = \frac{\Delta\phi}{\omega_{alinhamento}}$. A posição atual do movimento é controlada pela equação 7.

$$\phi(tis) = \begin{cases} \phi_i + \omega_{alinhamento} tis & \text{, se } tis \leq \Delta t \\ \phi_f & \text{, se } tis > \Delta t \end{cases} \tag{7}$$

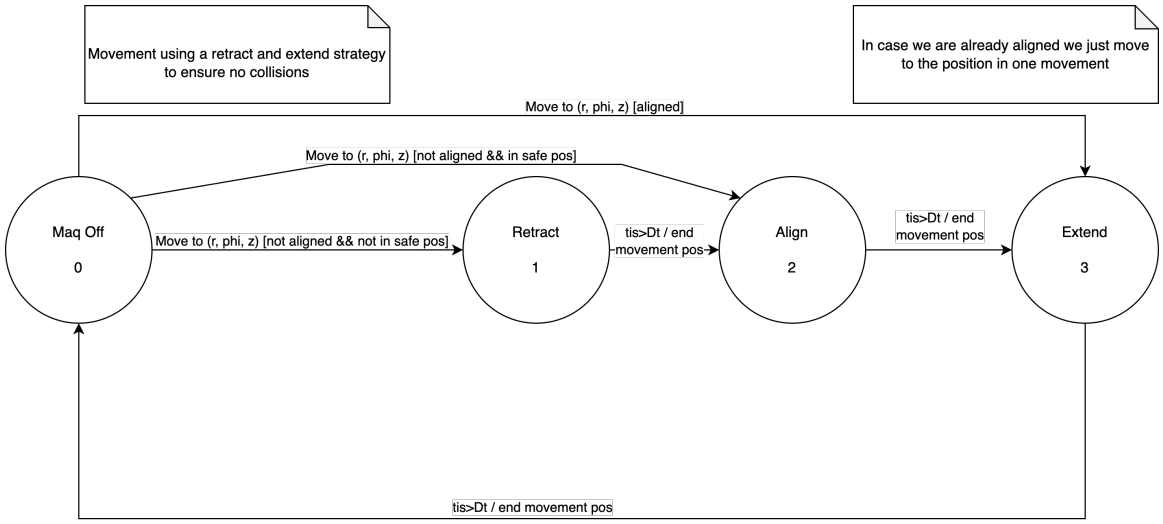

Figura 5: Máquina de Movimento Suave do Manipulador.

8 Máquina de *Pick and Drop*

De modo a executar a tarefa de *pick and drop* desenvolveu-se a máquina *Pick and Drop* representada na figura 6. Resumidamente, esta tarefa fecha a garra, apanha a peça, transporta a peça até à posição do sensor de cor, determina a cor da peça e conforme a cor da peça transporta a peça até ao respetivo local de depósito, abre a garra no local designado e por fim, mantendo o mesmo alinhamento, retrai até à posição segura. A máquina é ativada quando o evento *flag_pick_drop* é produzido.

De modo a abrir e fechar a garra determinou-se as duas posições do servomotor 3 que colocam a garra na posição aberta e fechada.

Quando a máquina *Pick and Drop* pretende realizar um movimento para uma determinada posição, esta simplesmente emite um evento para iniciar a tarefa do movimento suave.

Após a chegada à posição do sensor de cor, aguarda-se 500 ms de forma a estabilizar o movimento da garra em cima do sensor, garantindo assim uma leitura correta da cor da peça. A cor da peça é determinada conforme descrito na secção 4.1 e com base na cor lida a peça é transportada até ao respetivo depósito, onde a garra a larga. No final da tarefa, a garra retrai para uma posição segura mantendo o alinhamento com o depósito.

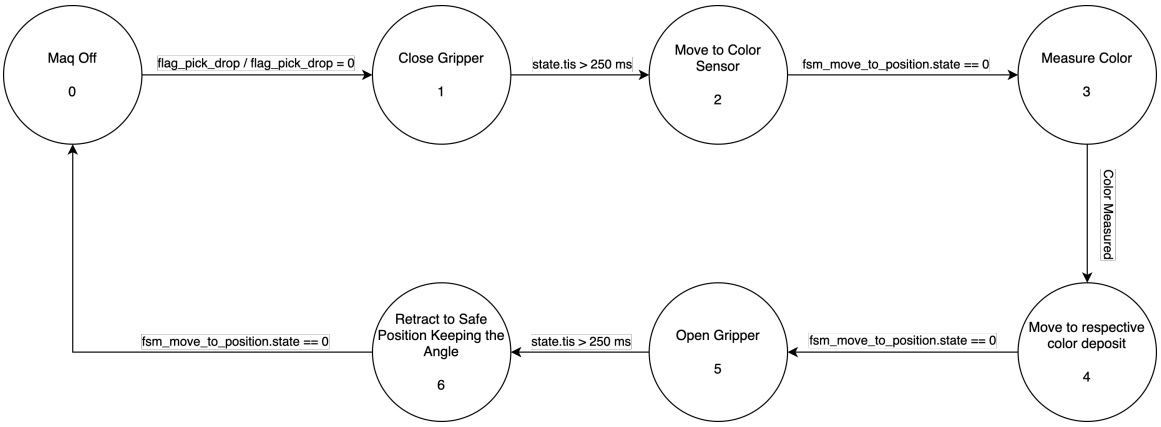

Figura 6: Máquina de *Pick and Drop*.

9 Máquina de recolha de uma matriz 3x3

De modo a ter a possibilidade de identificar as posições e pegar em 9 peças dispostas numa matriz 3x3, foi criada uma máquina de estados, representada na figura 7.

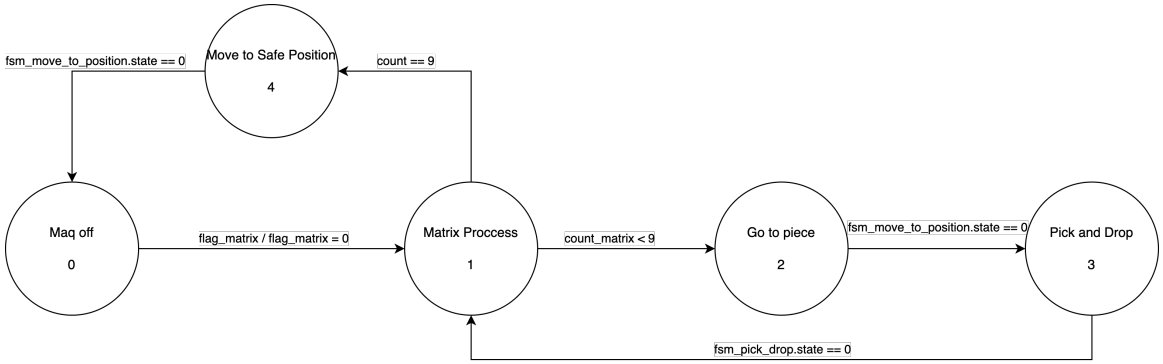

Figura 7: Máquina de Matriz.

As 9 posições consideradas são inicialmente definidas no código, pelo que o manipulador tem conhecimento, à partida, todas as posições a que terá de ir. Esta máquina é ativada inicialmente pela ocorrência do evento *flag_ matrix*. A partir daqui, o manipulador inicia o movimento ordenado de recolha de cada peça, fazendo um movimento linha a linha, da direita para a esquerda. Enquanto o contador de peças for inferior a 9, esta máquina irá oscilar entre os estados *Go to Piece*, dado que cada posição é conhecida, e *Pick and Drop*, ativando a máquina com esse nome para efetuar o movimento da peça, através da ocorrência da evento *flag_ pick_ drop*, para o reconhecimento da cor e armazenamento no local correto. Quando todas as peças tiverem sido recolhidas, estando o contador a 9, o manipulador move-se para a posição de segurança.

10 Máquina de varrimento com auxílio de sensor de tempo de voo

De forma a conseguir identificar e localizar uma peça cuja posição é previamente desconhecida, foi criada uma máquina de estados de forma a obter este funcionamento, representada na figura 8.

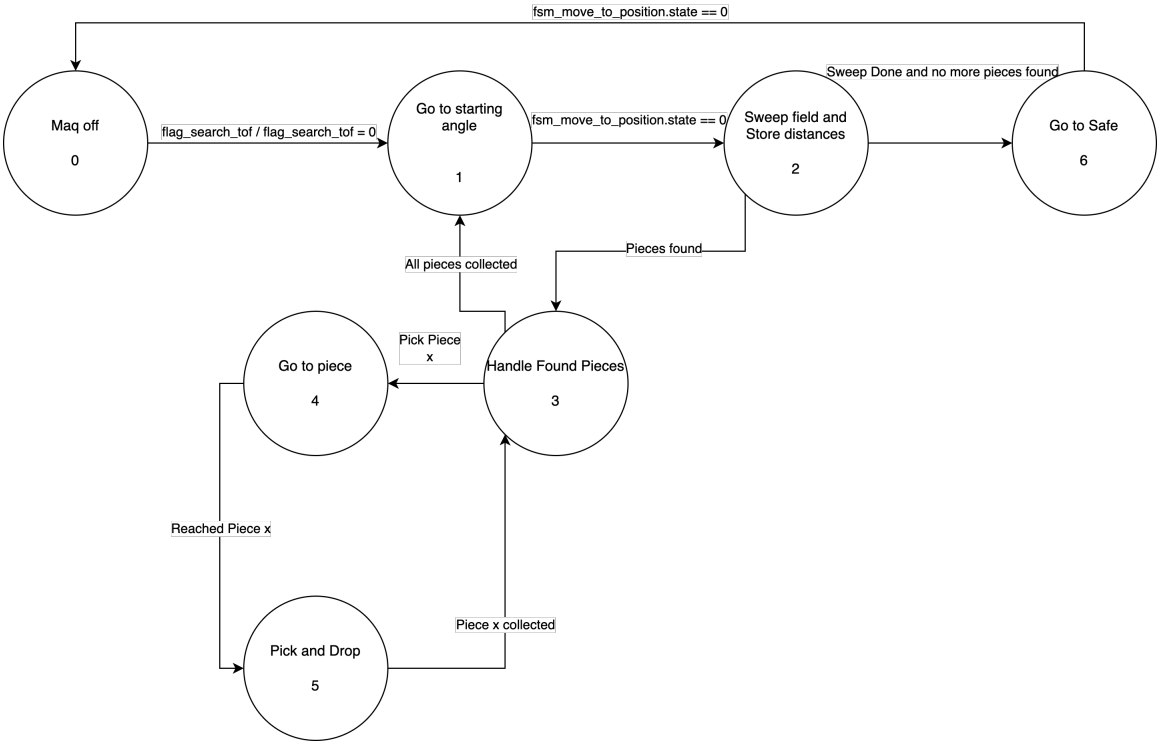

Figura 8: Máquina de TOF.

Esta tarefa utiliza o sensor TOF (*Time of Flight*), cuja implementação e funcionamento já foram previamente abordados na secção 4.2. Além disso, os ângulos limite em que o manipulador irá realizar o varrimento são definidos pelo utilizador como explicado na secção 5.1. Esta máquina é ativada quando se produz o evento *flag_ search_ tof*. Assim, no primeiro estado o manipulador irá para o ângulo de início definido e começará o varrimento do espaço, percorrendo ininterruptamente a janela angular definida, guardando as posições todas em que detetar uma peça. Quando este movimento for concluído, e caso tenham sido detetadas peças, esta máquina passa para um estado (*Handle Found Pieces*) onde, de forma cíclica, fará o movimento de aproximação e *Pick and Drop* até que todas as posições guardadas e respetivas peças tenham sido tratadas. Após isto, e por uma questão de redundância/segurança, o manipulador inicia um novo varrimento, de forma a verificar se todas as peças foram recolhidas. Em caso afirmativo, o manipulador move-se para a posição de segurança.

11 Estruturação Código e Implementação

A estrutura utilizada no código para as máquinas de estados consiste em duas secções principais: a primeira secção onde se tem em conta a lógica das transições e a segunda secção onde se realizam as ações dos estados. Além disso, na primeira secção verifica-se a lógica das transições e realizam-se as ações correspondentes às transições.

Para facilitar a organização do código, implementação e alteração de *features* desenvolveu-se várias bibliotecas que serão de seguida apresentadas. A primeira biblioteca chama-se *ServoMotorController* e contém o controlo direto dos servomotores, realizando a conversão de posição angular para *duty cycle* tendo em conta os valores de calibração. A biblioteca *InverseKinematics* contém as equações da cinemática inversa e armazena as posições relevantes para o funcionamento do manipulador. Esta biblioteca interage diretamente com a biblioteca dos servomotores sendo que assim o manipulador só utiliza esta biblioteca para realizar os movimentos. Por fim, desenvolveu-se a biblioteca *algorithms* onde se armazenaram as funções para o tratamento de dados proveninentes do sensor TOF.

Numa fase final, de forma a ter um sistema mais robusto, procurou-se implementar uma forma de guardar em memória no microcontrolador algumas estruturas de dados relevantes para o funcionamento do manipulador, permitindo assim o armazenamento de configurações novas sem necessitar de se reprogramar o microcontrolador. Alcançou-se isso através de um funcionamento análogo a uma EEPROM (que o RP2040 não possui, possui apenas uma memória *flash* de 2MB). Para tal, com recurso à biblioteca *EEPROM.h*, guardaram-se os seguintes dados:

- Posição do sensor de cor;
- Posições dos locais de depósito para cada cor;
- 9 posições da matriz 3x3;
- Ângulo mínimo/máximo do varrimento do TOF.

Para cada um destes dados, reservou-se apenas o espaço correspondente ao tamanho dos dados (utilizando a função *sizeof* em cada variável).

A cada *boot* do programa, este lê sempre o que esta guardado em memória (*EEPROM.get*) e sempre que existe uma alteração num dos dados guardados, essa é armazenada também na memória (*EEPROM.put*).

12 Conclusão

Concluindo, foi possível verificar que se atingiram todos os objetivos e modos de funcionamento pretendidos. Foi possível dotar o sistema de capacidades além do necessário para o seu correto funcionamento, como foi o caso dos dados em memória.

É também importante referir que o desenvolvimento deste trabalho permitiu um aprofundamento nos conhecimentos de microcontroladores, mais concretamente do RP2040 utilizado nas placas Raspberry Pi Pico, assim como o uso de diferentes sensores, com princípios de operação diferentes.

Além disso, referir que neste projeto se utilizaram conceitos ligados à cinemática inversa, que por um lado vieram simplificar bastante o movimento do manipulador, assim como permitiram uma aprendizagem sobre os mesmos.

Este projeto mostrou-se verdadeiramente gratificante na medida em que, tendo atingido os objetivos propostos, conseguiu atingir-se algo verdadeiramente funcional e útil.

Referências

[1] Frank Dellaert. *An Introduction to Serial Link Manipulators.* `https://piazza.com/class_profile/get_resource/hpa4u5hmxk599/hsdw3z0329m3st`. [Online; acedido a 4/02/2024].